AlphaChess-RL: a small AlphaZero-style chess engine

Midsem progress: exact endgames, search, and the supervised start

Vatsl Goswami Agriya Yadav Samyak Baid

14 October 2026

Reinforcement Learning course project · Ashoka University · Monsoon 2026

The question

Goal. Build a chess agent the AlphaZero way, at student scale:

1. a policy–value network learns from human games;
2. MCTS self-play improves it by reinforcement learning;
3. we measure how much Elo the RL adds on top of the supervised start.

Scope decisions (to fit our compute):

- self-play only from **French Defense** openings;
- games stop at $\leq$ **5 pieces** and take the exact tablebase result;
- the supervised stage still covers all of chess.

Three tracks

Vatsl: exact endgame DP, tablebases, Lichess data

Agriya: MCTS and the evaluation harness

Samyak: encoding, network, training

Weakness in non-French openings and in endgames the network never trains on is *expected*, not a bug.

Chess as an MDP

- **State** s: the position plus whose turn it is.
- **Action** a: a legal move. **Transition**: deterministic.
- **Reward**: only at the end: $+1$ win, 0 draw, -1 loss.
- Two players, zero-sum, so we store every value from the side to move's point of view.

One rule covers both players (**negamax Bellman**):

$$V(s) \;=\; \max_{a \in A(s)} \; -\gamma\, V\big(f(s,a)\big)$$

The minus sign means "switch seats": good for my opponent is bad for me.

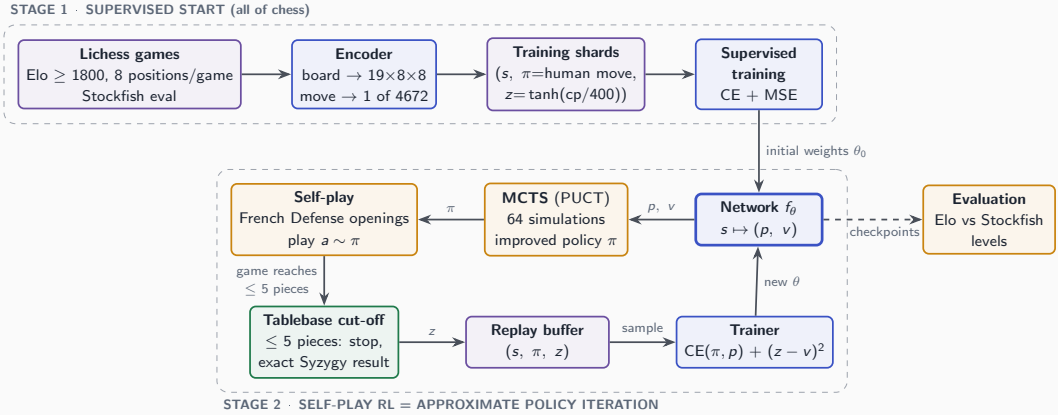
One convention, everywhere

Endgame tables, MCTS backups, the network's value head and the training targets all use *side-to-move* values in $[-1, 1]$.

The board encoding flips the board when Black moves, so the network always sees "my pieces moving up".

With $\gamma = 0.9$: mate in d plies is worth $\pm\gamma^d$, so the winner prefers faster mates and the loser longer resistance.

The whole pipeline



Colours: **network / training** · **search / evaluation** · **exact endgames** · **data**

Where we are (Phase 1)

Track	Done	Next
Exact endgame DP	KQK, KRK, KPK solved and verified 100% vs Syzygy; VI vs retrograde vs RTDP; approximation study	full 2M-position Lichess run
Lichess data	streaming, filtering, sampling, Parquet shards	
MCTS	batched PUCT, Gumbel root variant, UCI engine	Elo vs simulations with the real network
Evaluation	paired matches, Stockfish, Elo with bootstrap CIs, puzzle and centipawn-loss metrics	
Encoding	19 board planes, 4672-move index, legal mask	
Network	6-block ResNet, policy + value heads (505k params)	train on 2M positions
Training	supervised trainer; trial run on 5k positions works	

All parts meet at the shared interfaces (`docs/INTERFACES.md`): **402 tests pass**.

Exact endgames by dynamic programming · Vatsl

Endgames as a shortest-path problem

KQK, KRK, KPK: every position, both sides to move.

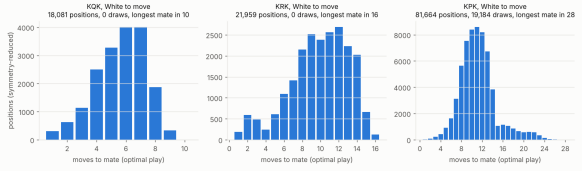
- **Value iteration**: back up every position every sweep until nothing changes. KRK: 33 sweeps, **1.65M** backups.
- **Retrograde analysis**: walk backwards from the mates, one DTM level at a time; each move is walked once. KRK: **0.52M** steps.
- **Symmetry**: 8 board symmetries (pawnless) shrink KQK/KRK to 81,920 slots instead of 524,288.

Tables store win/draw/loss + plies to mate (DTM) + the best move.

Table	Positions	Longest mate
KQK	46,137	mate in 10
KRK	50,015	mate in 16
KPK	165,676	mate in 28

Solve time: KQK 3.0 s, KRK 2.8 s, KPK 11.1 s on one CPU core. All three longest mates match the known values.

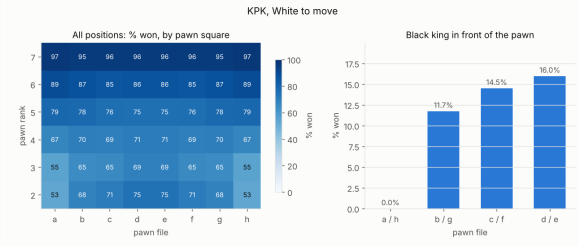
Exact tables, fully verified



Verification, every legal position:

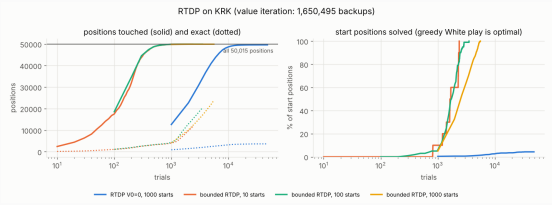
- win/draw/loss agrees with **Syzygy 100%** on all three tables;
- every value satisfies the negamax Bellman equation;
- an independent value iteration reproduces WDL and DTM exactly;
- Stockfish (1,000 positions per table) never found a shorter mate.

KPK: the textbook results fall out of the DP



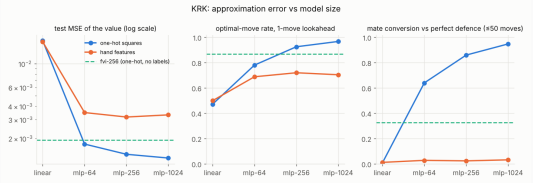
- White to move wins **76.5%** of KPK positions.
- The pawn's rank matters most: 53–75% won on rank 2, 96–97% on rank 7.
- **Rook pawn, defender in front: 0% won**, against 12–16% for b-, c-, d-pawns. Exactly the textbook rule.

RTDP: only back up what play actually reaches



- **Finding**: plain RTDP with $V_0 = 0$ does not converge on KRK. 0 is pessimistic for White (every position is won), so White stops exploring.
- **Fix**: keep an upper and a lower bound per position, so both sides stay optimistic. Converges with 37–41% of value iteration's backups.
- On a graph this small RTDP still touches almost every position. Its savings need a bigger space or narrow starts: exactly our French-only self-play setting.

How well can a network learn an exact value? (KRK)



Model	Inputs	Optimal move	Mate conv.
MLP-64	one-hot	78.1%	64.0%
MLP-1024	one-hot	96.8%	**94.8%**
MLP-1024	hand features	70.5%	3.2%
FVI-256 (no labels)	one-hot	86.9%	32.8%

- **Raw inputs + capacity beat hand features**: features know *who* wins, not *how*. This is the bet behind our big network.
- Bootstrapping without labels (fitted VI) drifts far from the mates.
- Errors compound over long mates: one wrong move in 16 loses the win.

Search and evaluation · Agriya

MCTS with PUCT

Each simulation:

1. **Select** down the tree by

$$Q(a) + c_{\text{puct}}\, P(a)\, \frac{\sqrt{N_{\text{parent}} + 1}}{1 + N(a)}$$

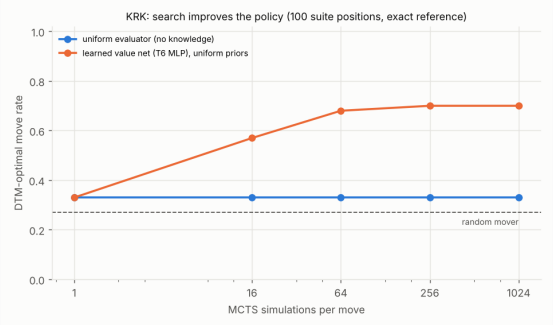
2. **Expand** a new position: the network gives priors P and value v.
3. **Evaluate**: rules for mate/stalemate, else v.
4. **Back up** to the root, negating at every move.

The root's visit counts $\pi(a) \propto N(a)$ are the **improved policy**.

Engineering

- batched leaf evaluation (GPU-friendly)
- legal priors only; malformed priors fall back to uniform
- visit counts sum exactly to the budget
- root Dirichlet noise for self-play; Gumbel root variant for low budgets
- tested: mate-in-one, backup signs, batching, terminal handling

Search turns an approximate value into a better policy



100 KRK positions, scored against the **exact** tables (DTM-optimal move rate):

Simulations	1	64	1024
Uniform evaluator	33%	33%	33%
Learned value (MLP-64)	33%	68%	70%

- With no knowledge, search can't see 10–32-ply mates: it stays flat.
- With an imperfect learned value, search doubles the optimal-move rate.
- This is AlphaZero's **policy-improvement step**, checked against ground truth.

Evaluation harness: measuring Elo honestly

- **UCI engine**: our agent speaks the standard chess-engine protocol.
- **Paired matches**: every opening played twice, colours swapped.
- **Stockfish** at fixed Elo levels (1350, 1500, 1700, 1900).
- **Elo** $= 400 \log_{10} \frac{\text{score}}{1-\text{score}}$ with 95% bootstrap intervals over *opening pairs*.
- Extra diagnostics: top-move agreement, centipawn loss, Lichess puzzles.

Elo vs simulations (0, 1, 16, 64, 256)

same checkpoint, openings, seeds

runs with the first trained network

Encoding, network and training · Samyak

Board encoding: what the network sees

A position becomes a stack of **19 planes of 8×8**:

Planes	Meaning
0–5	my P, N, B, R, Q, K
6–11	opponent's P, N, B, R, Q, K
12	repetition (seen before?)
13–16	castling rights, mine and theirs
17	colour to move
18	fifty-move counter / 100

Always the side to move's view

If Black is to move, the board is **mirrored top-to-bottom** and colours swapped. So "my pieces" are always planes 0–5 and always move "up".

One network for both colours, and its value output is automatically from the mover's side, matching the negamax convention.

Example: after 1. e4, Black's pawns appear on "rank 2" of planes 0, and White's e4-pawn appears on "rank 5" of plane 6.

Move encoding: 4672 = 64 from-squares × 73 move types

$$\text{index} = \text{from_square} \times 73 + \text{move_type}$$

- **0–55**: queen-like, 8 directions × 7 distances (also king, pawn, castling, queen promotion)
- **56–63**: the 8 knight jumps
- **64–72**: underpromotions to N, B, R × 3 directions

Illegal entries are masked to $-\infty$ before the softmax.

Worked examples (start position)

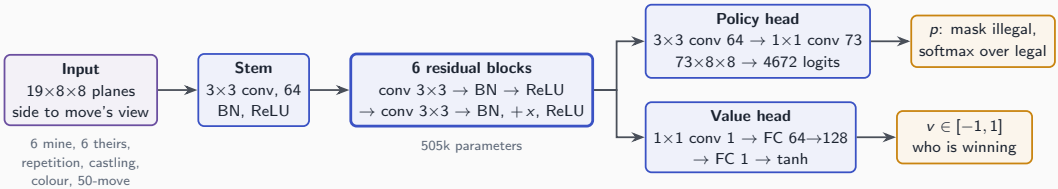
e2e4: from e2 = square 12, north (dir 0), distance 2 ⇒ type 1
index = $12 \times 73 + 1 = \mathbf{877}$

g1f3: from g1 = square 6, knight jump $(-1, +2)$ = type 63
index = $6 \times 73 + 63 = \mathbf{501}$

1. . . e5 for Black is *also* 877, because the board is flipped.

Tested: every legal move in 308 positions maps to a unique index and back.

The policy–value network



Policy head is convolutional: plane t at square s is the logit of move type t from s. Flattening gives index $s \cdot 73 + t$, exactly the move encoding.

Value head squashes to $[-1, 1]$ with tanh: the expected result for the side to move. Plugs straight into Agriya's MCTS (tested).

Supervised training on Lichess

Data (Lichess, January 2024):

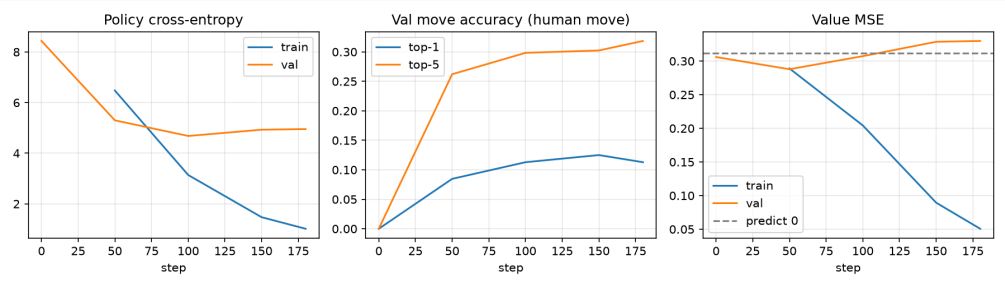
- rated games, both players ≥ 1800, no bullet, with Stockfish evals;
- 8 positions per game after move 4, split train/val/test by game;
- target move π = the human's move (one-hot);
- target value $z = \tanh(\text{cp}/400)$, mate $\to \pm 1$, side to move's view.

Loss (AlphaZero's, weight decay via AdamW):

$$\mathcal{L} = \underbrace{-\sum_a \pi(a) \log p_\theta(a \mid s)}_{\text{policy cross-entropy}} + \underbrace{(z - v_\theta(s))^2}_{\text{value MSE}}$$

Shards stream one at a time, so memory stays flat as the dataset grows. Checkpoints, metrics and curves are written automatically.

Trial run: the training loop works end to end



4,456 train / 248 val positions (625 games), 6×64 ResNet, 10 epochs, **25 s on a MacBook Air GPU**.

Val top-1 **0%** → **11%**, top-5 **0%** → **32%** (random legal move ≈ 3%).

As expected at this size: the net memorises (train CE 1.1 vs val 5.0) and the value head does not beat predicting 0 yet (0.33 vs 0.31). Judging who is winning needs far more data ⇒ the 2M-position run.

Methodology going forward

Step 1: supervised training at scale

- **2M positions** from Lichess 2024-01 ($\approx 450\times$ the trial run), encoded with the same pipeline.
- Train on the shared **2×H100** workstation; same trainer, same checkpoints.
- Watch validation curves for overfitting; scale the network (e.g. 10 blocks $\times$ 128 channels) once the data supports it.
- **Deliverables**: val move accuracy, value error vs the predict-0 baseline, and the first **Elo vs simulations** plot with Agriya's harness.

Why start supervised?

A random network makes MCTS no better than chance (the flat 33% line). Human games give a sensible starting policy and value, so self-play starts from real chess.

Step 2: self-play RL as approximate policy iteration

Repeat:

1. **Policy improvement (search)**: MCTS from each position, guided by (p_θ, v_θ), returns visit counts π, a *better* policy than p_θ.
2. **Play**: self-play games from French Defense openings, moves sampled from π.
3. **Policy evaluation (outcome)**: each game ends in a result z; games reaching ≤ 5 pieces take the *exact* tablebase value.
4. **Training stores the improvement**: fit $p_\theta \to \pi$ and $v_\theta \to z$ on the replay buffer.

Policy iteration, with a network

Classic PI alternates evaluation and greedy improvement over a table.

Here the "greedy step" is search, and the network is a compressed, approximate table, so it is **approximate** PI (Bertsekas & Tsitsiklis).

Vatsl's KRK result shows the improvement step working: 33% $\to$ 70%.

The tablebase cut-off gives exact z in endgames and shortens games.

Evaluation plan

Three curves

1. **Elo vs Stockfish levels** 1350 / 1500 / 1700 / 1900 for the supervised and the RL network.
2. **Elo vs MCTS simulations** 0, 1, 16, 64, 256 with a fixed checkpoint.
3. **Elo vs training iterations** during self-play: does RL keep adding Elo?

Protocol

- frozen checkpoint, openings, seeds, Stockfish settings
- pilot 20–40 games, then 200 games per point
- colour-swapped pairs; 95% bootstrap intervals
- no claims where intervals overlap
- supporting: puzzles, top-move agreement, centipawn loss

Ablations

Ablation	What it tells us
Supervised only vs supervised + RL	the headline: how much Elo self-play adds
MCTS budget in self-play (16 / 64 / 256)	how much search quality drives the RL gain
Tablebase cut-off on vs off	does exact endgame adjudication help learning?
Value target: game result z vs blended with search Q	noise vs bias in value targets
Network size (6×64 vs 10×128)	capacity vs data
PUCT vs Gumbel root at low budgets	better policy improvement with few simulations?
French-only vs held-out openings	how far the RL gains transfer

Next steps

Vatsl

- full 2M-position Lichess run and encoding
- replay buffer, self-play workers, tablebase adjudication

Agriya

- Elo vs simulations with the trained network
- rate every RL checkpoint in the same harness

Samyak

- long supervised run on the H100s
- RL trainer: (s, π, z) from the replay buffer
- checkpoint schedule for self-play

Summary

The three pieces are built and connected: exact endgames (verified 100%), batched MCTS with an honest Elo harness, and an encoder + network + trainer that learns on real data. Next: train at scale, then let search teach the network.

Questions?